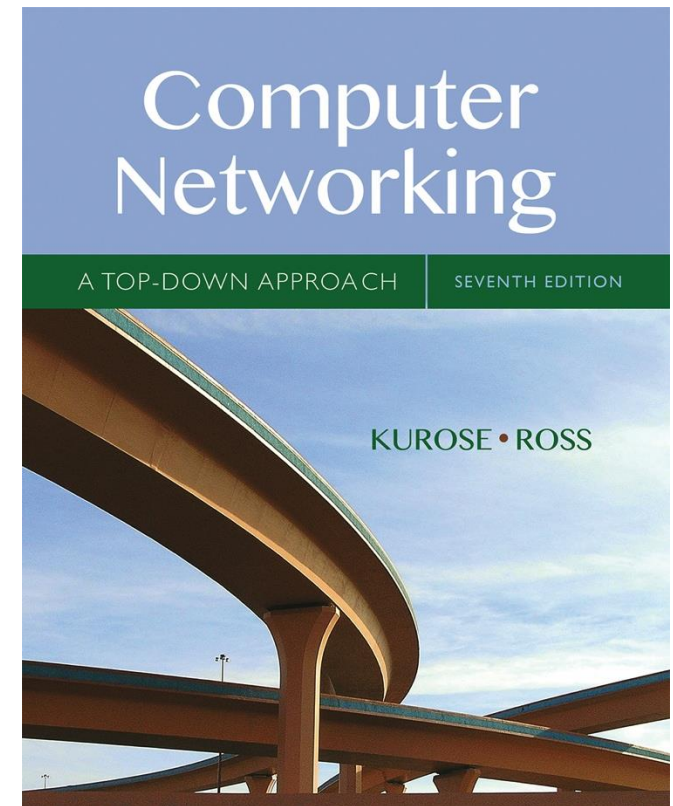


Chapter 9 Multimedia Networking

Merk: litt overlapp med
kapittel fra ny bok

Multimedia networking	Fra Chapter 9 er KUN FØLGENDE pensum: - 9.1 Multimedia networking applications - 9.2 Streaming stored video - 9.3 Voice-over-IP - 9.4 Protocols for real-time conversational applications - 9.4.1 RTP (men ikke 9.4.2 SIP)
-----------------------	--



*Computer
Networking: A Top
Down Approach*

7th edition

Jim Kurose, Keith Ross

Pearson/Addison Wesley

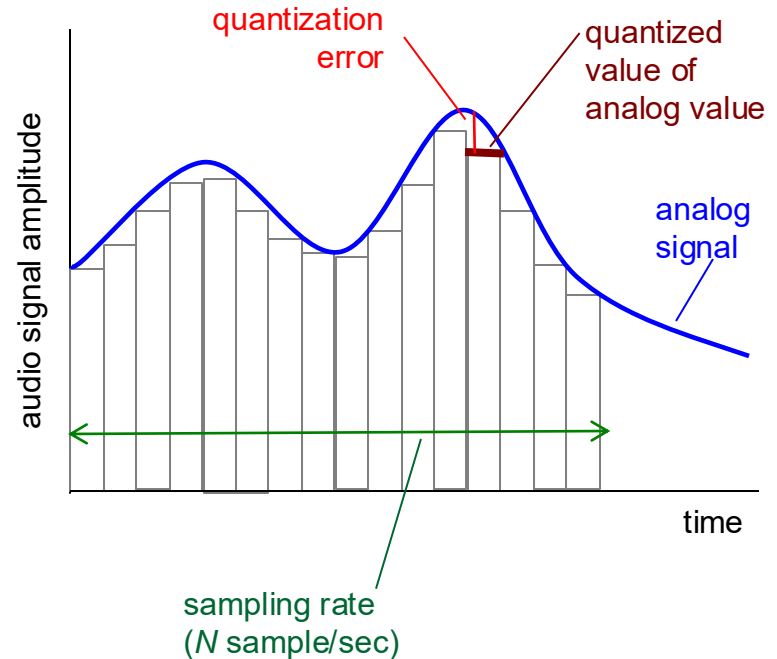
April 2016

Multimedia networking: outline

- 9.1 multimedia networking applications
- 9.2 streaming *stored* video
- 9.3 voice-over-IP
- 9.4 protocols for *real-time* conversational applications
- 9.5 network support for multimedia

Multimedia: audio

- analog audio signal sampled at constant rate
 - telephone: 8,000 samples/sec
 - CD music: 44,100 samples/sec
- each sample quantized, i.e., rounded
 - e.g., $2^8=256$ possible quantized values
 - each quantized value represented by bits, e.g., 8 bits for 256 values

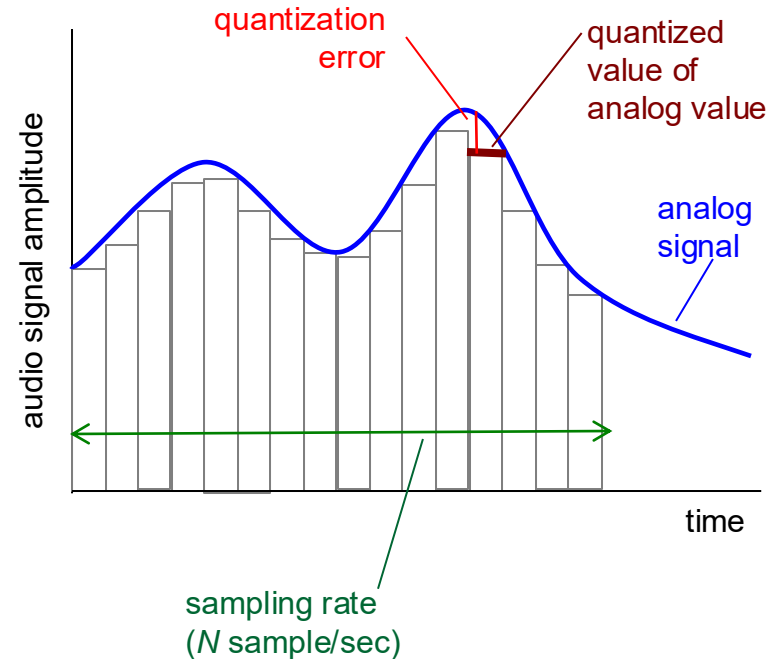


Multimedia: audio

- example: 8,000 samples/sec, 256 quantized values: 64,000 bps
- receiver converts bits back to analog signal:
 - some quality reduction

example rates

- CD: 1.411 Mbps
- MP3: 96, 128, 160 kbps
- Internet telephony: 5.3 kbps and up



Multimedia: video

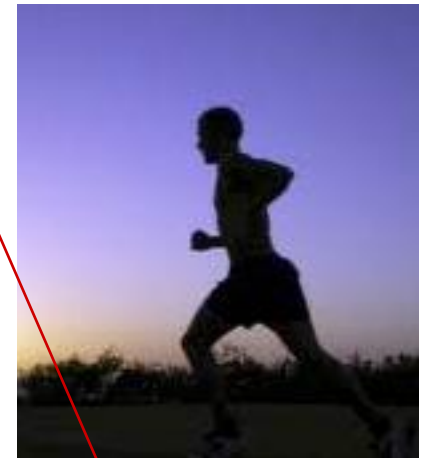
- video: sequence of images displayed at constant rate
 - e.g., 24 images/sec
- digital image: array of pixels
 - each pixel represented by bits
- coding: use redundancy *within* and *between* images to decrease # bits used to encode image
 - spatial (within image)
 - temporal (from one image to next)

spatial coding example: instead of sending N values of same color (all purple), send only two values: color value (*purple*) and number of repeated values (N)



frame i

temporal coding example: instead of sending complete frame at $i+1$, send only differences from frame i



frame $i+1$

Multimedia: video

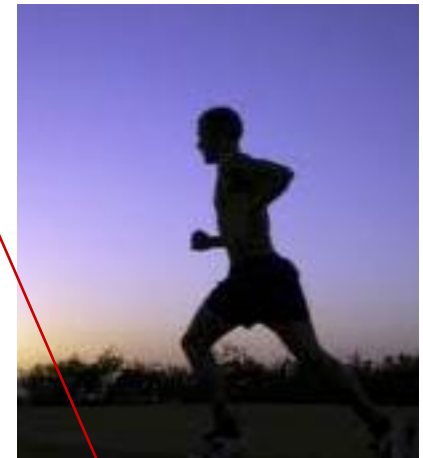
- **CBR: (constant bit rate):**
video encoding rate fixed
- **VBR: (variable bit rate):**
video encoding rate changes
as amount of **spatial**,
temporal coding changes
- **examples:**
 - MPEG I (CD-ROM) 1.5 Mbps
 - MPEG2 (DVD) 3-6 Mbps
 - MPEG4 (often used in Internet, < 1 Mbps)

spatial coding example: instead of sending N values of same color (all purple), send only two values: color value (*purple*) and number of repeated values (N)



frame i

temporal coding example: instead of sending complete frame at $i+1$, send only differences from frame i



frame $i+1$

Multimedia networking: 3 application types

- *streaming, stored* audio, video
 - *streaming*: can begin playout before downloading entire file
 - *stored (at server)*: can transmit faster than audio/video will be rendered (implies storing/buffering at client)
 - e.g., YouTube, Netflix, Hulu
- *conversational* voice/video over IP
 - interactive nature of human-to-human conversation limits delay tolerance
 - e.g., Skype
- *streaming live* audio, video
 - e.g., live sporting event (futbol)

Multimedia networking: outline

9.1 multimedia networking applications

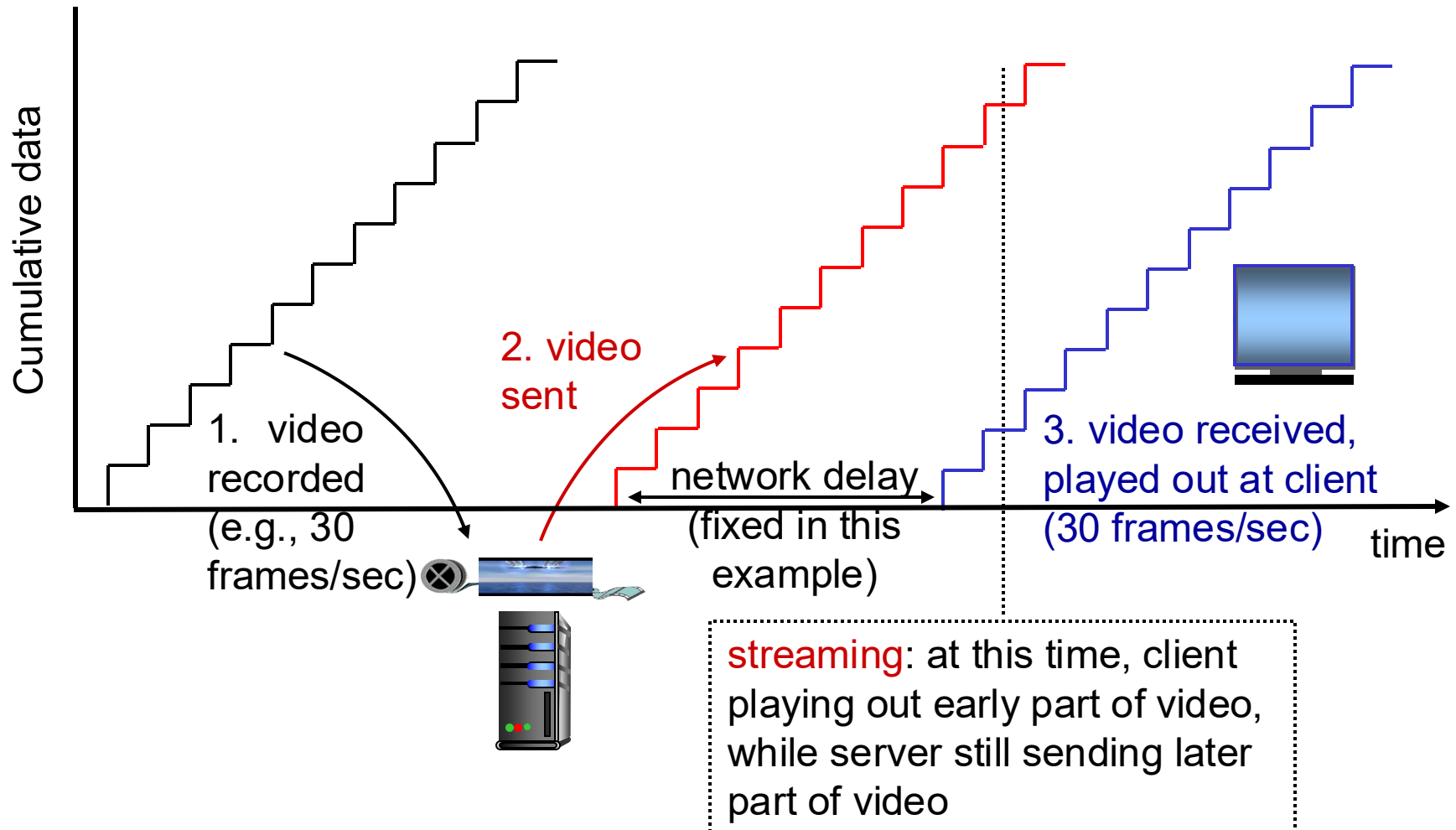
9.2 *streaming stored video*

9.3 voice-over-IP

9.4 protocols for *real-time* conversational applications

9.5 network support for multimedia

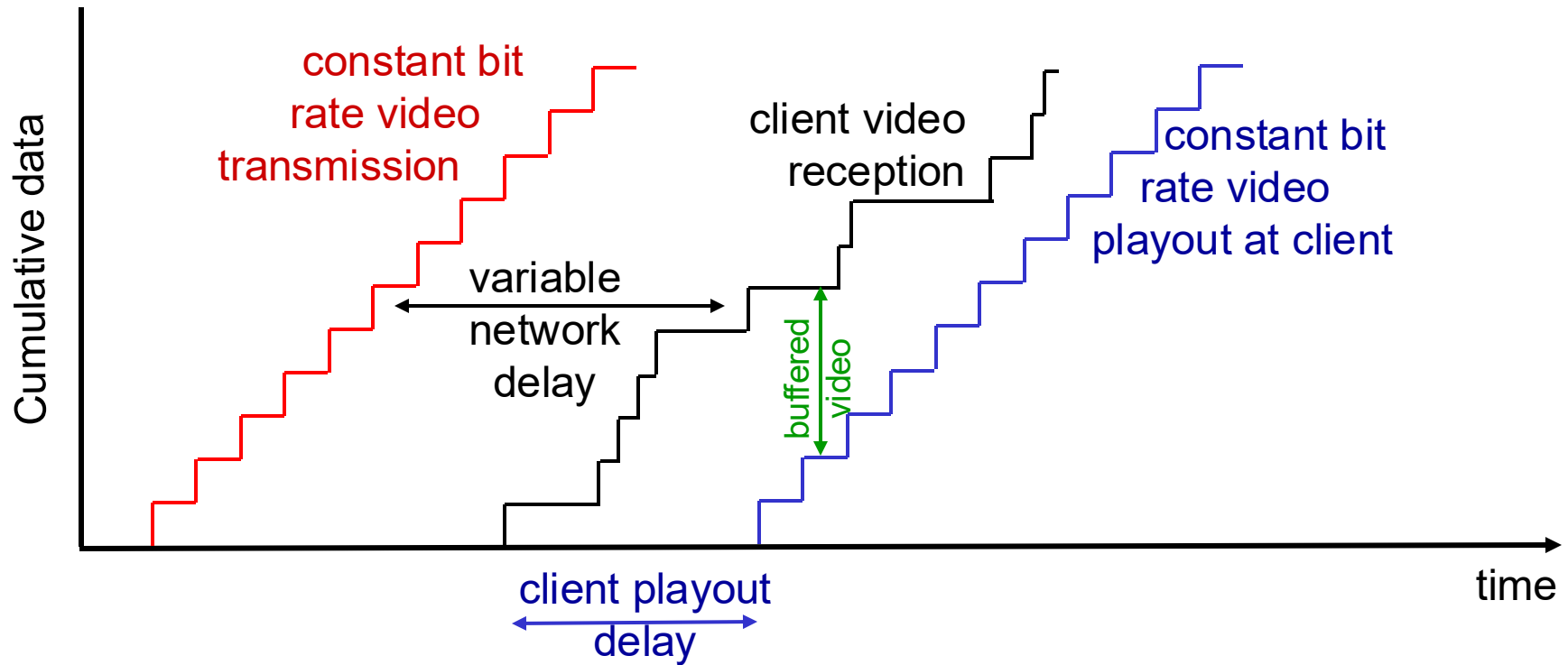
Streaming stored video:



Streaming stored video: challenges

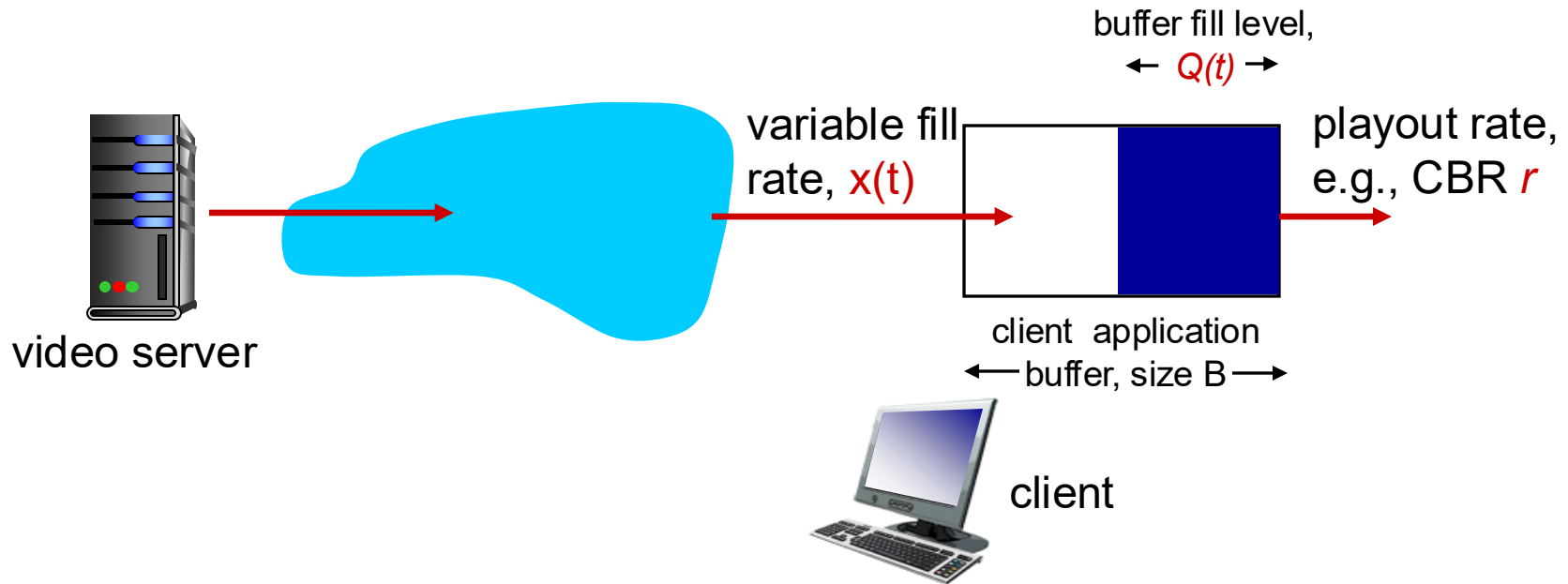
- **continuous playout constraint**: once client playout begins, playback must match original timing
 - ... but **network delays are variable** (jitter), so will need **client-side buffer** to match playout requirements
- other challenges:
 - client interactivity: pause, fast-forward, rewind, jump through video
 - video packets may be lost, retransmitted

Streaming stored video: revisited

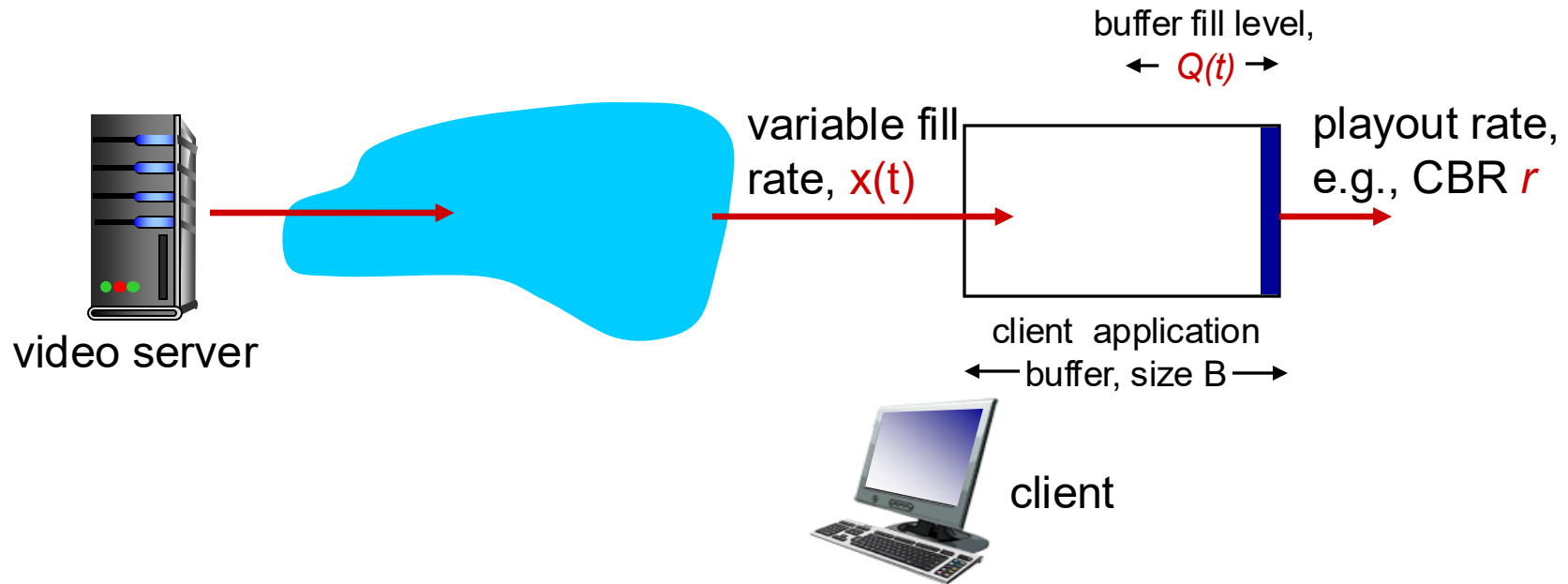


- *client-side buffering and playout delay*: compensate for network-added delay, delay jitter

Client-side buffering, playout

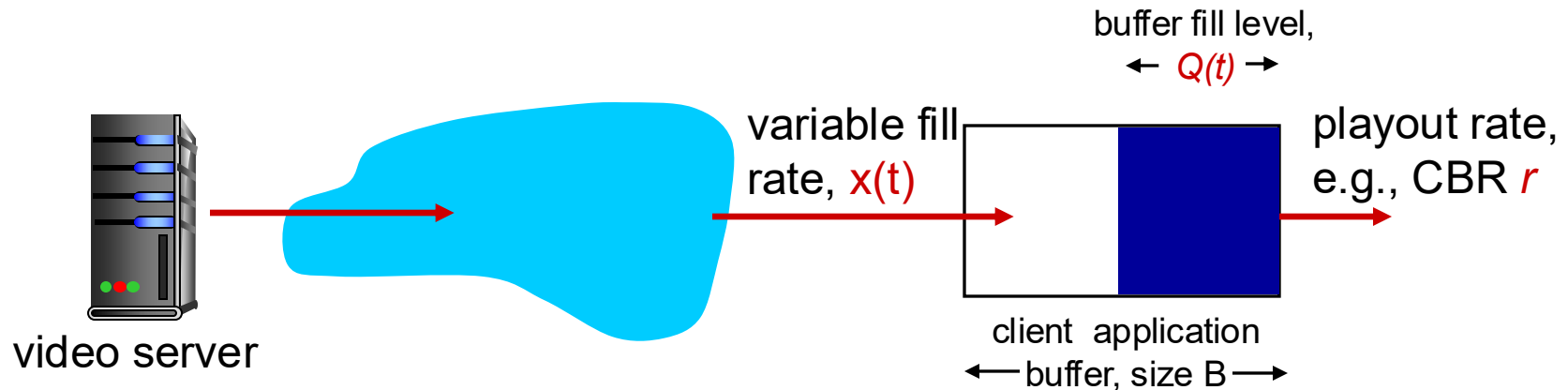


Client-side buffering, playout



1. Initial fill of buffer until playout begins at t_p
2. playout begins at t_p ,
3. buffer fill level varies over time as fill rate $x(t)$ varies and playout rate r is constant

Client-side buffering, playout



playout buffering: average fill rate (x), playout rate (r):

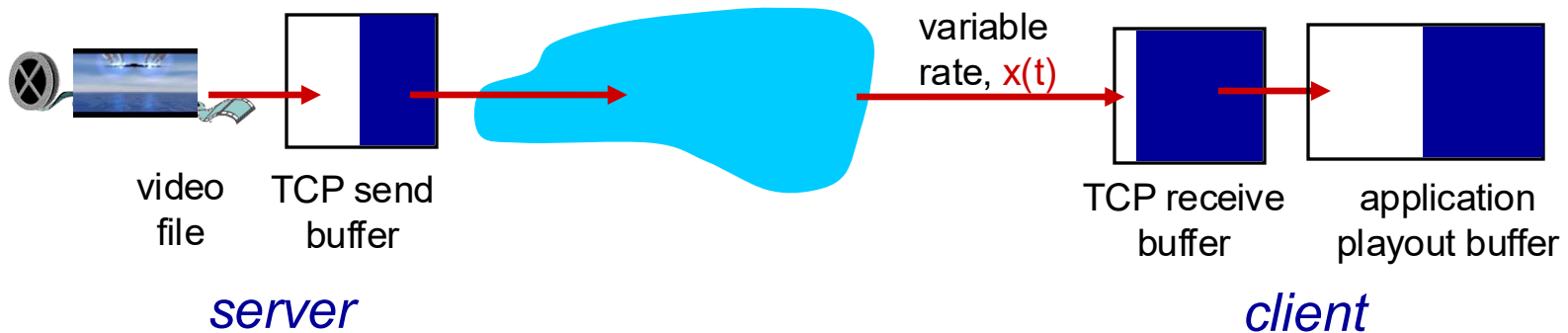
- $x < r$: buffer eventually empties (causing freezing of video playout until buffer again fills)
- $x \geq r$: buffer will not empty, provided initial playout delay is large enough to absorb variability in $x(t)$
 - *initial playout delay tradeoff*: buffer starvation less likely with larger delay, but larger delay until user begins watching

Streaming multimedia: UDP

- server sends at rate appropriate for client
 - often: send rate = encoding rate = constant rate
 - transmission rate can be oblivious (*uvitende*) to congestion levels
- short playout delay (2-5 seconds) to remove network jitter
- error recovery: application-level, time permitting
- RTP [RFC 2326]: multimedia payload types
- UDP may *not* go through firewalls (*hvorfor ?*)

Streaming multimedia: HTTP

- multimedia file retrieved via HTTP GET
- send at maximum possible rate under TCP



- fill rate fluctuates due to TCP congestion control, retransmissions (in-order delivery)
- larger playout delay: smooth TCP delivery rate
- HTTP/TCP passes more easily through firewalls